Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы оптимизации и управления»

ОТЧЕТ
к лабораторной работе №1
на тему:
«ОБРАЩЕНИЕ МАТРИЦЫ С ИЗМЕНЕННЫМ СТОЛБЦОМ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

(дата, подпись студента)

Проверил доцент каф. ИКТ
ГОРБУКОВ Дмитрий Михайлович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Выполнение работы 4
Вывод 5
Список использованных источников 6
Приложение А (справочное) Исходный код 7

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Реализовать алгоритм обращения матрицы с измененным столбцом: по известным матрицам A и A^{-1}, вектору x и индексу i определить, обратима ли матрица A’, полученная заменой i-го столбца матрицы A на x, и в случае обратимости найти A'^{-1}.

2 ВЫПОЛНЕНИЕ РАБОТЫ

В программе реализован пошаговый алгоритм из методических указаний. Сначала вычисляется вектор $l = A^{-1} x$, затем формируются векторы l_tilde и l_hat, после чего строится вспомогательная матрица Q и находится итоговая обратная матрица по формуле $A'^{-1} = Q A^{-1}$.

Для проверки использованы данные из файла lab1_input.json.

После выполнения алгоритма получены следующие промежуточные и итоговые результаты:

```
l = [1, 0, 1]
l_i = 1
l_tilde = [1, 0, -1]
l_hat = [-1, 0, 1]
Q = [
  [1, 0, -1]
  [0, 1, 0]
  [0, 0, 1]
]
A_bar = [
  [1, -1, 1]
  [0, 1, 0]
  [0, 0, 1]
]
(A_bar)^-1 = [
  [1, 1, -1]
  [0, 1, 0]
  [0, 0, 1]
]
```

Так как l_i != 0, измененная матрица является обратимой. Дополнительная проверка прямым обращением дает тот же результат:

```
(A_bar)^-1 (прямая проверка) = [
  [1, 1, -1]
  [0, 1, 0]
  [0, 0, 1]
]
```

```
Исходная матрица A:[[ 1 -1  0]
 [ 0  1  0]
 [ 0  0  1]]
Обратная матрица A^-1:[[1 1 0]
 [0 1 0]
 [0 0 1]]
Вектор-столбец x:[1 0 1]
Индекс столбца для замены i = 3
Матрица A_bar (A с замененным 3-м столбцом):[[ 1 -1  1]
 [ 0  1  0]
 [ 0  0  1]]
Шаг 1: Вектор l = A^-1 * x:
[1 0 1]

l[2] = 1 != 0, матрица A_bar обратима.
Шаг 2: Вектор l' (l_prime):[ 1  0 -1]
Шаг 3: Вектор l_hat:[-1. -0.  1.]
Шаг 4: Матрица Q:[[ 1.  0. -1.]
 [ 0.  1. -0.]
 [ 0.  0.  1.]]
Шаг 5: Итоговая обратная матрица (A_bar)^-1 = Q * A^-1:[[ 1.  1. -1.]
 [ 0.  1.  0.]
 [ 0.  0.  1.]]
--- Проверка результата ---
Обратная матрица, вычисленная с помощью np.linalg.inv(A_bar):[[ 1.  1. -1.]
 [ 0.  1.  0.]
 [ 0.  0.  1.]]

Результаты сохранены в output.json
```

Рисунок 2.1 – Результат работы программы

ВЫВОД

В ходе лабораторной работы был реализован алгоритм быстрого пересчета обратной матрицы при замене одного столбца исходной матрицы. Для тестового примера установлено, что матрица A’ обратима, а вычисленная матрица A'^{-1} совпадает с результатом прямой проверки. Это подтверждает корректность реализованного метода.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Python Documentation [Электронный ресурс]. – Режим доступа: https://docs.python.org/. – Дата доступа: 09.04.2026.

[2] NumPy Documentation [Электронный ресурс]. – Режим доступа: https://numpy.org/doc/. – Дата доступа: 09.04.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Файл входных данных lab1_input.json

```
{
  "A": [
    [1, -1, 0],
    [0, 1, 0],
    [0, 0, 1]
  ],
  "A_inv": [
    [1, 1, 0],
    [0, 1, 0],
    [0, 0, 1]
  ],
  "x": [1, 0, 1],
  "i": 3
}
```

Листинг А.2 – Реализация обращения матрицы с измененным столбцом

```
import numpy as np
import json

def invert_modified_matrix(A_inv, x, i):
    n = A_inv.shape[0]
    if i < 1 or i > n:
        raise ValueError(f"Индекс i должен быть в диапазоне [1, {n}]")

    # Переводим i в 0-based индекс
    i_zero_based = i - 1

    # Шаг 1: Находим вектор l = A^-1 * x
    l = np.dot(A_inv, x)
    print(f"Шаг 1: Вектор l = A^-1 * x:\n{l}\n")

    if np.isclose(l[i_zero_based], 0):
        print(f"l[{i_zero_based}]   =   {l[i_zero_based]},   матрица   A_bar
необратима.")
        return None
    else:
        print(f"l[{i_zero_based}]  =  {l[i_zero_based]}  !=  0,  матрица  A_bar
обратима.")

    # Шаг 2: Формируем вектор l_prime
    l_prime = l.copy()
    l_prime[i_zero_based] = -1
    print(f"Шаг 2: Вектор l' (l_prime):{l_prime}")

    # Шаг 3: Находим вектор l_hat
    l_hat = (-1 / l[i_zero_based]) * l_prime
    print(f"Шаг 3: Вектор l_hat:{l_hat}")

    # Шаг 4: Формируем матрицу Q
    Q = np.identity(n)
    Q[:, i_zero_based] = l_hat

    print(f"Шаг 4: Матрица Q:{Q}")
```

```
    # Шаг 5: Находим (A_bar)^-1 = Q * A^-1
    A_bar_inv = np.dot(Q, A_inv)
    print(f"Шаг 5: Итоговая обратная матрица (A_bar)^-1 = Q * A^-1:{A_bar_inv}")

    return {
        "l": l.tolist(),
        "li": float(l[i_zero_based]),
        "l_prime": l_prime.tolist(),
        "l_hat": l_hat.tolist(),
        "Q": Q.tolist(),
        "A_bar_inv": A_bar_inv.tolist()
    }

def main():
    with open('lab1_input.json', 'r') as f:
        data = json.load(f)

    A = np.array(data['A'])
    A_inv = np.array(data['A_inv'])
    x = np.array(data['x'])
    i = data['i']

    print(f"Исходная матрица A:{A}")
    print(f"Обратная матрица A^-1:{A_inv}")
    print(f"Вектор-столбец x:{x}")
    print(f"Индекс столбца для замены i = {i}")

    A_bar = A.copy()
    A_bar[:, i-1] = x
    print(f"Матрица A_bar (A с замененным {i}-м столбцом):{A_bar}")

    results = invert_modified_matrix(A_inv, x, i)

    if results is not None:
        print("--- Проверка результата ---")
        numpy_inv = np.linalg.inv(A_bar)
        print(f"Обратная матрица, вычисленная с помощью
np.linalg.inv(A_bar):{numpy_inv}")

        output_data = {
            "A_bar": A_bar.tolist(),
            "A_bar_inv": results["A_bar_inv"],
            "l": results["l"],
            "li": results["li"],
            "l_tilde": results["l_prime"],
            "l_hat": results["l_hat"],
            "Q": results["Q"]
        }

        with open('output.json', 'w') as f:
            json.dump(output_data, f, indent=2)
        print("\nРезультаты сохранены в output.json")

if __name__ == "__main__":
    main()
```